定义题目

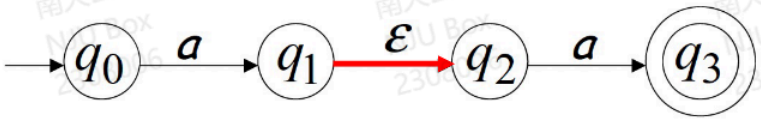
# Deterministic Finite Automata

- A DFA is a five-tuple: $(Q, \Sigma, \delta, q_0, F)$
  - $Q$: A finite set of states
  - $\Sigma$: A finite set of input characters, i.e., an alphabet
  - $\delta: Q \times \Sigma \longmapsto Q$: Transition function, e.g., $\delta(q, a) = q'$
  - $q_0 \in Q$: The start state
  - $F \subseteq Q$: A finite subset of final states

- An extension of transition: $\delta^*: Q \times \Sigma^* \longmapsto Q$
  - $\delta^*(q, abba) = q'$; $\delta^*(q, \epsilon) = q$;

# $\epsilon$-Transition

- Transition to the next states without reading any inputs




南大云盘 NJU Box 2308006

# Definition of ~~DFA~~NFA

- A ~~DFA~~NFA is a five-tuple: $(Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$
  - $Q$: A finite set of states
  - $\Sigma$: A finite set of input characters, i.e., an alphabet
  - $\delta: Q \times (\Sigma \cup \{\epsilon\}) \longmapsto \cancel{Q} 2^Q$: Transition function, e.g., $\delta(q, a) = \{q', q''\}$
  - $q_0 \in Q$: The start state
  - $F \subseteq Q$: A finite subset of final states

# Regular Language

- Take a DFA $M = (Q, \Sigma, \delta, q_0, F)$, language can be accepted by the DFA is written as $L(M) = \{w \in \Sigma^*: \delta^*(q_0, w) \subseteq F\}$

- Take a NFA $M = (Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$, language can be accepted by the NFA is written as $L(M) = \{w \in \Sigma^*: \delta^*(q_0, w) \cap F \neq \emptyset\}$

$L$ is a regular language if there's a DFA/NFA $M$ such that $L = L(M)$

南大云盘 NJU Box 2308006

# Context Free Grammar (CFG)

- A context-free grammar is a tuple $G = (N, T, S, P)$
  - $N$: a finite set of non-terminals
  - $T$: a finite set of terminals, such that $N \cap T = \emptyset$
  - $S \in N$: start non-terminals
  - $P$: production rules in the form of $A \rightarrow a$, where $A \in N$ and $a \in (N \cup T)^*$

assignment → identifier = expression
expression → term + term
term → identifier
term → identifier * number

assignment → identifier = expression
expression → term + term
term → identifier
| identifier * number

# Predictive Parsing

- Predictive parsers are recursive descent parser w/o backtracking
- LL(1)
  - L: scanning input from left to right
  - L: leftmost derivation
  - 1: Using one input symbol of lookahead at each step

- LL(1) grammar (Not ambiguous! Not left-recursive!)
  - Rich enough to cover most programming constructs

# First() and Follow()

- $\mathrm{FIRST}(\alpha)$:
  - A set of terminals that $\alpha$ may start with
- $\mathrm{FOLLOW}(\alpha)$:
  - A set of terminals that can appear immediately to the right of $\alpha$

# Static Single-Assignment

- **Feature 1**: Every variable has only one definition
- **Feature 2**: Using φ to merge definitions from multi paths
- => Direct def-use chains

# Dominance Relations

- A **dom** B
  - if all paths from Entry to B goes through A
- A **post-dom** B
  - if all paths from B to Exit goes through A

- Strict (post-)dominance – A (post-)**dom** B but A ≠ B

- Immediate dominance – A strict-dom B, but there's no C, such that A strict-dom C, C strict-dom B

# Dominator Tree

- Almost linear time to build a dominator tree.
  - Node: Block
  - Edge: Immediate dom relation

# Dominance Frontier

- DF(B) = { … } for the block B
  - The immediate successors of the blocks dominated by B
  - Not strictly dominated by B

# Data Flow Analysis

- To perform data flow analysis, define
  - transfer function f for each statement/block
  - merge function ∧ at the joint point of multiple paths
  - the initial set of data flow facts for each block

- Data flow analysis produces
  - IN[B]/OUT[B] that include data flow facts at a program point
  - The resulting data flow facts are always true during program execution

## Reaching Definitions

- A definition *d* may reach a program point, if there is a path from *d* to the program point such that *d* is not killed along the path.

## Available Expressions

- An expression x + y is available at a point p if
  - every path from the entry node to p evaluates x + y, and
  - after the last such evaluation prior to reaching p, there are no subsequent assignments to x or y.

## Live Variable Analysis

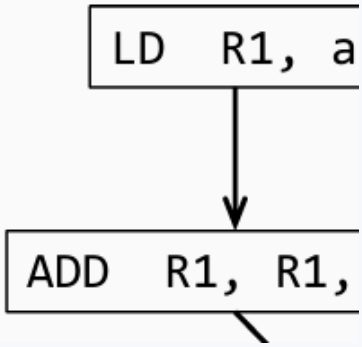
- We wish to know if the value of a variable x at point p can be used along the path starting from p.

## DPLL

- Davis–Putnam–Logemann–Loveland (DPLL) algorithm
- **Goal**: Find a solution to satisfy the input formula

# In-Block Scheduling

- True/Fake Dependence
- Dependence Graph
- List Scheduling



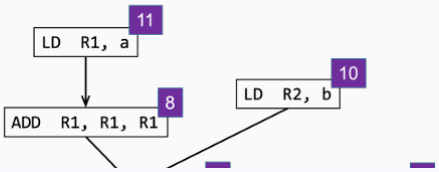

## Why Scheduling?

- Every modern high-performance processor can execute several operations in a single clock cycle.

## Why Compilers Matter?

- Hardware only can execute instructions that have been fetched
- Hardware has limited space to buffer stalled operations

- Compilers can place independent operations close together to allow better hardware utilization

# List Scheduling

- Check each clock cycle
- Schedule instructions when they are ready
- When multi instructions can be scheduled, check their **priority**
  - The longest latency path (max depth)
  - Using the most resources
  - …